

Remaining Setup — Your Side

Everything that can be built in code is done and merged to `main` (and pushed to GitHub). What remains needs **your accounts, credentials, devices, and dashboards** — things I can't do for you. This guide sequences those steps. Work top to bottom: each step unblocks the next.

Legend: RELEASE-BLOCKER needed for the iOS App Store release ENABLES BUILD unblocks more app work CONFIG dashboard/secret only OPTIONAL future enhancement

At a glance — the order

#	Step	Why	Detailed runbook
1	Finish EAS dev builds	Foundation: lets you test everything native on real devices	<code>docs/superpowers/push-setup.md</code>
2	Enable Apple sign-in	Mandatory for the iOS release (App Store 5.1.1)	<code>docs/superpowers/5i-social-signin-setup.md</code>
3	Enable Google sign-in	Already coded; needs OAuth credentials	<code>docs/superpowers/5i-social-signin-setup.md</code>
4	Wire up push delivery	Turns push notifications on; enables the B4b polish	<code>docs/superpowers/push-setup.md</code>
5	Production email (Resend)	Blast + roster-CSV emails actually send	<code>docs/superpowers/email-delivery-setup.md</code>
6	Production SMTP (auth emails)	OTP / password-reset / change-email emails	this guide
7	On-device verification pass	Confirm the gated features actually work	this guide
8	Production hardening (C6)	<code>double_confirm_changes</code> for destructive ops	this guide

Step 1 — Finish the EAS dev builds ENABLES BUILD

You already ran `eas init` (project `app.padeljam` is linked; `projectId` is committed). Now produce installable dev clients on your two phones. A dev build (not Expo Go) is required for the native modules — Apple sign-in, push, chat, image capture, geocoding.

Do — from `apps/mobile/` :

```
eas build --profile development --platform ios
eas build --profile development --platform android
# When prompted "non-exempt encryption?" it won't ask anymore — already declared exempt.
# Install each finished build on the matching device, then start the JS dev server:
npx expo start --dev-client
```

Verify

- Both dev clients install and launch on the physical devices.
- The app boots, you can sign in (phone OTP), and chat connects.

Monorepo note: EAS installs deps from the workspace root via the committed `pnpm-lock.yaml`. If a cloud build fails at the install step, send me the error — there's a small EAS-monorepo config fix.

Step 2 — Enable Apple sign-in RELEASE-BLOCKER

The app code (native Apple button, token exchange, duplicate-email conflict handling) is done. You need to register the Apple identifiers and turn the provider on. **Required before the iOS release** because the app offers Google sign-in (App Store guideline 5.1.1).

Do

- **Apple Developer console:** on the App ID for `app.padeljam`, enable the *Sign in with Apple* capability. Create a **Services ID** and a **Sign in with Apple key** (.p8).
- **Supabase** → **Authentication** → **Providers** → **Apple:** enable it; set the Client ID (your Services ID / bundle id) and the secret.
- **If using local/CLI config** (`infra/supabase/config.toml` → `[auth.external.apple]`): set `enabled = true`, `client_id`, and provide the secret via env var `SUPABASE_AUTH_EXTERNAL_APPLE_SECRET`.

Verify (on the iOS dev build)

- "Continue with Apple" shows the native sheet → you land signed in.
- A brand-new Apple account → routed to "Create your account" with name pre-filled.
- An Apple email that already belongs to another account → inline "already linked" message, bounced to sign-in.

Step 3 — Enable Google sign-in CONFIG

Google sign-in is already coded; it just needs OAuth credentials wired in. Runbook: `docs/superpowers/5i-social-signin-setup.md`.

Do

- **Google Cloud console:** create an OAuth 2.0 client; add the Supabase callback + `mobile://auth/callback` as authorized redirects.
- Set the secrets (Supabase dashboard or function/auth env):

```
SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_ID=<your-client-id>
SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_SECRET=<your-client-secret>
```

- Enable the Google provider in Supabase (or `[auth.external.google]` `enabled = true`).

Verify

- "Continue with Google" completes the browser handshake and signs you in (new → create-account; returning → home).

Step 4 — Wire up push delivery

ENABLES BUILD

CONFIG

Token registration + the send-push function + the DB trigger are all built (and a delivery log records each attempt). Push is a **no-op** until you supply provider credentials and the fan-out settings. Runbook: `docs/superpowers/push-setup.md`.

Do

1. Upload push credentials (interactive):

```
cd apps/mobile && eas credentials # add FCM (Android) + APNs (iOS) keys
```

2. Deploy the function:

```
pnpm dlx supabase@latest --workdir infra functions deploy send-push --no-verify-jwt
```

3. Set the shared secret (pick a strong random value):

```
pnpm dlx supabase@latest --workdir infra secrets set PUSH_WEBHOOK_SECRET='<strong-random>'
```

4. Point the DB trigger at the function (in the Supabase SQL editor / prod DB):

```
ALTER DATABASE postgres SET app.send_push_url = '<send-push function URL>';  
ALTER DATABASE postgres SET app.send_push_secret = '<same value as PUSH_WEBHOOK_SECRET>';
```

`app.send_push_secret` must equal `PUSH_WEBHOOK_SECRET`. Never commit either.

Verify (on a device)

- ☐ Grant notification permission on first launch; a push token registers.
- ☐ Trigger a notification (e.g. have someone follow you / invite you) → the push arrives.
- ☐ Tap it → the app deep-links to the right screen (event / group / community / profile).

After this works, ping me: I'll build **B4b** — app-icon badge counts + Expo delivery receipts + retry-on-failed — which only makes sense to build once live push is verifiable.

Step 5 — Production email via Resend

CONFIG

The blast email + "email me the roster CSV" paths call Resend; they return `email_not_configured` until keyed. Runbook: `docs/superpowers/email-delivery-setup.md`.

Do

- Create a Resend account; verify your sender domain (e.g. `padeljam.app`).
- Set the function secrets:

```
pnpm dlx supabase@latest --workdir infra secrets set RESEND_API_KEY=re_XXXXXXX  
pnpm dlx supabase@latest --workdir infra secrets set "RESEND_FROM_EMAIL=Padel Jam <noreply@padeljam.app>"
```

Verify

- Send a blast (email channel) / "email me the roster CSV" → the email arrives; the blast's delivery badge shows *Delivered*.

Step 6 — Production SMTP for auth emails CONFIG

Supabase's *auth* emails (OTP fallback, password reset, change-email confirmations) need a real SMTP sender in production (the local Inbucket mailbox is dev-only).

Do — in Supabase → Project Settings → Auth → SMTP (or `config.toml` `[auth.email.smtp]`): set host, port, user, and the password via an env var (e.g. SendGrid/Resend SMTP). Set a verified from-address.

Verify

- Trigger a password reset / email-change → the message is delivered from your domain.

Step 7 — On-device verification pass RELEASE-BLOCKER

Once Steps 1–4 are in place, run through the features that could only be verified on a real dev build. Tell me about anything that misbehaves and I'll fix it.

Checklist

- **Apple sign-in** — new / returning / conflict (relay email) cases.
- **Google sign-in** — full handshake + the conflict case.
- **Push tap-routing** — tap a notification in foreground, background, and from a cold start → correct screen.
- **Result image-card** — finish an event → Share Results → "Share externally" produces the standings image.
- **Location geocoding** — type a venue address → "Find location from address" → ✓ coords; the event shows a distance in Explore.
- **Chat** — archive/delete, the media gallery paginates past 100 messages, and list rows update live as messages arrive.
- **Thumbnails** — pick an event image at create/edit; it uploads + displays.

Step 8 — Production hardening (C6) CONFIG

Destructive account actions use `double_confirm_changes`, kept `false` locally for convenience. For production, enable it so email/phone changes require confirmation on both the old and new address.

Do — set `double_confirm_changes = true` in the production auth config (Supabase dashboard / `config.toml` `[auth.email]`).

Optional / future (no rush) OPTIONAL

- **Venue-catalog coordinates + proximity venue search** — store coordinates on the curated `venues` catalog and let users search venues by nearness. (Today: events get coordinates via on-device geocoding; the catalog itself has none.) A real build slice when you want it — just ask.

- **Keyed Places autocomplete** — a Google/Mapbox typeahead place picker for slicker venue entry. Needs an API key + billing; the on-device geocoder already covers the core need without one.
- **WhatsApp blast channel** — descope'd; would need a Twilio WhatsApp Business approved sender.

Reference — where each secret is used in code

Secret / setting	Used by
<code>SUPABASE_AUTH_EXTERNAL_APPLE_SECRET</code> + Apple client_id	Apple sign-in provider (<code>config.toml</code> <code>[auth.external.apple]</code>)
<code>SUPABASE_AUTH_EXTERNAL_GOOGLE_CLIENT_ID/SECRET</code>	Google sign-in provider (<code>[auth.external.google]</code>)
<code>PUSH_WEBHOOK_SECRET</code> + <code>app.send_push_url</code> / <code>app.send_push_secret</code>	send-push function + <code>notify_push</code> trigger
<code>RESEND_API_KEY</code> / <code>RESEND_FROM_EMAIL</code>	send-blast + send-roster-csv functions
EAS push credentials (FCM / APNs)	Expo push delivery to devices

Detailed step-by-step runbooks live in `docs/superpowers/` : `5i-social-signin-setup.md` , `email-delivery-setup.md` , `push-setup.md` , `eas-dev-build-setup.md` . Never commit secrets — set them as Supabase function secrets / project env, or your local `.env` .